The database calls are essentially RPCs, and almost all standard database drivers (say, JDBC) use blocking I/O. So you're already in the state described earlier, using threads and RPCs together. The memory locks that synchronize threads and the database locks that protect against mutation of table records are not the same thing, and you have to take great care when combining them.

You went from one programming model to two that are interwoven. Your first rewrite of the application took a lot longer than expected.

This is a dramatization

The traditional approach to building the team chat app goes sour in a catastrophic way. Although the example is exaggerated, you've probably seen projects run into at least some of these problems (I have seen similar cases first-hand). To quote Dean Wampler from his presentation "Reactive Design, Languages, and Paradigms" (http://mng.bz/1q7n), "In reality, good people can make almost any approach work, even if the approach is suboptimal."

So, is this example project impossible to complete with the traditional approach? No, but it's suboptimal. It will be very hard to keep complexity low and flexibility high while the application scales.

1.4.2 Traditional scaling and interactive use: Polling

You run in this configuration for a while, and the number of users increases. The web application servers aren't using many resources; most are spent on (de-)serializing requests and responses. Most of the processing time is spent in the database. The code on the web server is mostly waiting for a response from the database driver.